Tutorial 4 Submission

AI-Driven Physical IoT Prototype: KS5002 Smart Soccer Robot

A polished one-file PDF that matches the course requirement: assembled photos, exact AI prompt/response used, and a short explanation of how the generated idea was improved on the real hardware.

6 hardware modules | **2 AI sections highlighted** | **3 adaptive edge policies**

GitHub Repository: https://github.com/3351666087/ks5002-soccer-robot-tutorial4

Prototype Summary

Scenario: a smart campus football-training robot that can move, detect distance, capture the ball, and execute a release-and-ram kick with lightweight edge intelligence.

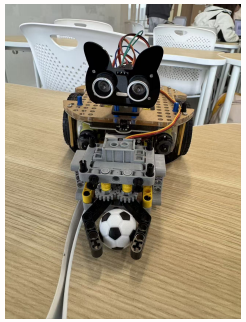
Modules used: ultrasonic sensor, pan servo, claw servo, dual DC motors, 8x8 LED matrix, RGB LEDs, and Wi-Fi HTTP control.

Evidence shown below: one fully assembled photo and one wiring/top-view photo from the physical prototype.

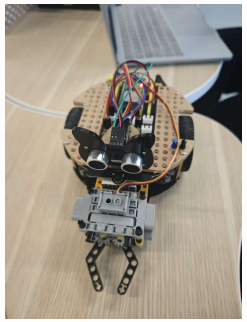
Exact AI Prompt Used

Generate a complete, ready-to-upload MicroPython control program for the Keyestudio KS5002 soccer robot for a smart campus football-training scenario. Use these modules: ultrasonic sensor, dual DC motors, head servo, claw servo, 8x8 LED matrix, RGB LEDs, and Wi-Fi HTTP control. Requirements: read sensor data in real time, trigger actuators on thresholds, display status on the matrix, support manual and auto modes, add simple edge intelligence using moving averages plus adaptive profile selection, keep the code modular, and include detailed comments.

Adapted from the course prompt template to the KS5002 robot hardware available in class.



Assembled kit



Wiring close-up

Highlighted AI Code

AI-generated section 1: `AdaptiveBandit` logic for scan/capture/ram profile choice and learning.

AI-generated section 2: HTTP button routing plus manual/auto command handling for `/btn/F`, `/btn/B`, `/btn/L`, `/btn/R`, `/btn/S`, `/btn/o`, `/btn/uNNN`, `/btn/vNNN`, `/btn/l`, `/btn/m`, and `/btn/n`.

Applied + Improved

Human improvements after testing:

- tuned the real claw and head servo geometry for this robot;
- added station-first Wi-Fi with AP fallback and `/status` monitoring;
- persisted learned policy values and refined reward rules using physical trials.

Code evidence: soccer_bot.py, main.py, config.py, remote_controller.py | GitHub repo highlighted on this page

Appendix

Exact Prompt and AI Response

This page keeps the wording explicit so the submission shows exactly what was asked from the AI and what parts were carried into the final prototype.

Prompt

```
Generate a complete, ready-to-upload MicroPython
control program for the Keyestudio KS5002 soccer
robot for a smart campus football-training
scenario. Use these modules: ultrasonic sensor,
dual DC motors, head servo, claw servo, 8x8 LED
matrix, RGB LEDs, and Wi-Fi HTTP control.
Requirements: read sensor data in real time,
trigger actuators on thresholds, display status on
the matrix, support manual and auto modes, add
simple edge intelligence using moving averages
plus adaptive profile selection, keep the code
modular, and include detailed comments.
```

AI Response

Build the solution in three layers so it stays debuggable on real hardware.

1. Hardware layer: initialize the motors, ultrasonic sensor, head servo, claw servo, RGB LEDs, and the HT16K33 matrix. Add helper functions for filtered distance reads, servo writes, motion, and face updates.

2. Decision layer: create a lightweight `AdaptiveBandit` with `choose()` and `learn()`. Use one bandit for scan profiles, one for capture profiles, and one for ram-kick profiles. Encode nearby distance context in the state key and keep epsilon-greedy exploration enabled.

3. Control layer: expose manual and automatic behaviour. Manual mode should support `/btn/F`, `/btn/B`, `/btn/L`, `/btn/R`, `/btn/S`, `/btn/o`, `/btn/uNNN`, `/btn/vNNN`, `/btn/l`, `/btn/m`, and `/btn/n`. Automatic mode should scan, avoid obstacles, capture the ball, and then perform a release-and-ram kick.

Recommended runtime logic:
- reverse and turn if the front distance is below the obstacle threshold;
- reward capture or ram actions when free space improves after the action;
- persist learned values to `robot_policy.json`;
- use the matrix to show `idle`, `search`, `capture`, `ram`, `celebrate`, and `obstacle`.

Recommended debug improvements:
- tune claw/head servo offsets on the real robot;
- add station Wi-Fi first, with AP fallback;
- expose a `/status` endpoint for remote monitoring.

How the Response Was Applied or Improved

We applied the AI structure, but improved it after hardware testing. The final project split the policy into `scan`, `capture`, and `ram` agents, persisted the learned table, added a station-first Wi-Fi snapshot plus AP fallback, expanded HTTP button aliases, and tuned the claw/head servo settings to fit the actual KS5002 wiring on our robot.

Post-Class

250-Word Reflection on AI Assistance

The course asked for a short reflection comparing AI-assisted development with manual coding. The text below is ready to submit as part of the report package.

faster first draft | more structured debugging | human tuning still needed

Reflection

AI assistance changed the development process in two clear ways: speed and decision quality. If we had written the whole robot controller manually, we would have spent much longer just building the first working version of the motion logic, servo control, and HTTP command mapping. With AI, we were able to generate a structured starting point much faster and move directly into testing on the physical kit. That mattered because the hardest problems were not purely coding problems. They appeared only after we uploaded the program and observed how the real robot behaved.

The most useful part of AI support was not “writing everything for us”, but giving us a fast draft that we could adapt. The AI response suggested a modular design with separate hardware, decision, and control layers. That made it easier to debug the project step by step. For example, we could test the ultrasonic scan, then the claw sequence, then the ram-kick behaviour, instead of changing the whole file blindly. AI also helped us think about lightweight edge intelligence, so we introduced adaptive profile selection rather than a fixed one-size-fits-all action.

At the same time, manual improvement was still essential. The generated idea had to be tuned for our actual robot: the claw servo angle, the head center offset, the Wi-Fi fallback behaviour, and the reward rules all needed human adjustment. In other words, AI reduced the time needed to reach a strong prototype, but the final quality still depended on our debugging, calibration, and understanding of the hardware. Compared with manual coding alone, AI made the workflow more iterative, faster to start, and more focused on testing real behaviour.